

Отчёт: Паромная переправа (вариант 28)

Oct 8, 2026 · @gleb

Титульные данные

- **Исполнитель:** Ри Ярослав Андреевич
- **Группа:** БПИ2510
- **Вариант:** 28 — «Паромная переправа»
- **Язык:** C (gnu11), только libc и системные вызовы POSIX (open, read, dprintf, sigaction, usleep)
- **Сборка:** make (gcc, флаги -Wall -Wextra -O2), Linux или WSL
- **Объём:** около 700 строк в 15 файлах, плюс 7 тестовых конфигураций и скрипт запуска

Условие задачи

28. Паромная переправа. Между двумя берегами реки ходит автомобильный паром. Автомобили разных типов прибывают на оба берега и встают в отдельные очереди. У каждого автомобиля есть уникальный номер, масса и число занимаемых мест.

Паром ограничен по грузоподъёмности и по числу перевозимых автомобилей. Погрузка идёт только у берега и прекращается, если очередной автомобиль нельзя принять без нарушения хотя бы одного ограничения. Паром отправляется после полного заполнения, по истечении максимального времени ожидания либо по решению диспетчера, если дальнейшая погрузка невозможна. После погрузки аппарат закрыывается, паром пересекает реку и разгружается; во время движения погрузка и выгрузка запрещены. Затем паром принимает автомобили на другом берегу.

Может быть предусмотрен приоритет автомобилей экстренных служб, но стратегия не должна приводить к неограниченному ожиданию обычных автомобилей. Паром, диспетчер и автомобили — самостоятельные участники модели.

Входные параметры: грузоподъёмность и вместимость парома; число автомобилей на каждом берегу; масса и размер автомобилей; интервалы прибытия; время погрузки, выгрузки и движения; максимальное время ожидания; стратегия формирования рейса; правила обслуживания приоритетных автомобилей.

Отображаемые события: прибытие автомобилей, формирование очередей, начало и окончание погрузки, отказ в погрузке, отправление, прибытие парома, выгрузка, текущее заполнение.

Инварианты: лимиты парома не превышены; автомобиль находится либо на берегу, либо на пароме; движение начинается только после завершения погрузки и закрытия аппарата; погрузка и выгрузка — только у соответствующего берега; автомобиль перевозится не более одного раза.

Завершение: после перевозки всех автомобилей, после заданного числа рейсов или по прерыванию пользователем (режим без ограничения по времени). Неперевезённые автомобили попадают в итоговую статистику.

Сценарий: сущности и процессы

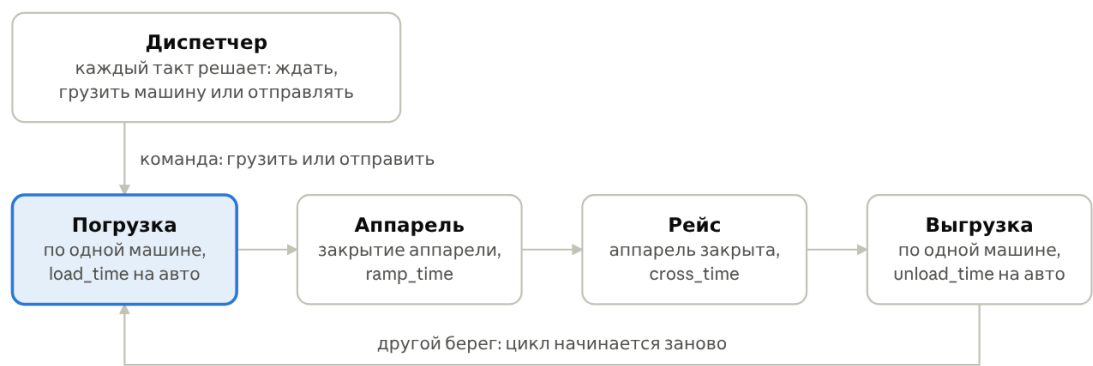
Модель дискретная: время измеряется тактами (1 такт = 1 минута), а один проход главного цикла — это один такт. В каждом такте по порядку выполняются четыре действия: приезд новых машин, шаг паром (внутри него диспетчер принимает решения), проверка инвариантов, вывод событий с паузой `delay_ms`. Паром, диспетчер и автомобили — отдельные участники, у каждого свой модуль и своё состояние.

Сущность из условия	Модуль	Как отображается в программе
Автомобиль	<code>car.c</code>	Структура <code>Car</code> : номер, масса, число мест, признак экстренной, берег, положение (<code>LOC_QUEUE</code> → <code>LOC_FERRY</code> → <code>LOC_DONE</code>). Параметры случайны в заданных границах; 60% машин занимают 1 место, 20% — 2, 10% — 3, 10% — 4
Очередь на берегу	<code>car.c</code>	Массив номеров машин для каждого берега. Обычная машина встаёт в конец, экстренная — вперёд, но не обгоняет тех, кто ждёт дольше <code>aging_limit</code>
Паром	<code>ferry.c</code>	Конечный автомат: погрузка → закрытие аппарели → рейс → выгрузка. Хранит борт, массу, занятые места, таймер операции, признак открытой аппарели
Диспетчер	<code>dispatcher.c</code>	Каждый такт погрузки отвечает на один вопрос: ждать, грузить конкретную машину или отправлять паром (с указанием причины)
Приход машин	<code>car.c</code>	<code>arrivals_step</code> : на каждом берегу следующая машина появляется через случайный интервал <code>arr_min...arr_max</code>
Проверка модели	<code>stats.c</code>	<code>check_invariants</code> после каждого такта и итоговая

Сущность из условия	Модуль	Как отображается в программе
		статистика

Сценарий одного рейса. Паром стоит у берега с открытой аппарелью. Диспетчер смотрит на первую машину очереди: если она помещается по массе, местам и числу машин, паром грузит её (load_time тактов); если нет — в журнал пишется отказ, и в зависимости от стратегии диспетчер берёт другую машину либо даёт команду на отправление. Паром также уходит, когда заполнен, когда истёк max_wait, когда очередь пуста и новых машин на этом берегу больше не будет. После закрытия аппарели (ramp_time) паром идёт cross_time тактов, причаливает к другому берегу, выгружает машины по одной (unload_time на каждую) и снова начинает погрузку — уже на противоположном берегу.

Диспетчер вмешивается только на погрузке, остальные состояния идут по таймерам



цикл парома · 4 состояния, 1 решающий участник

Из состояния погрузки паром выходит только по решению диспетчера; остальные три состояния длятся ровно столько тактов, сколько задано параметрами, и не требуют решений.

Структура программы и стратегии

Программа разбита на семь модулей; всё состояние модели лежит в одной структуре world (world.h), которую модули передают друг другу по указателю.

Файл	Строк	Назначение
main.c	78	разбор аргументов, обработка сигналов, главный цикл, освобождение памяти
config.c/.h	114	значения по умолчанию, чтение файла настроек, проверка параметров
car.c/.h	105	создание машин, очереди,

Файл	Строк	Назначение
		приоритет экстренных
ferry.c/.h	176	автомат парома, погрузка, рейс, выгрузка
dispatcher.c/.h	69	решения диспетчера и стратегии
log.c/.h	52	вывод на экран и в файл через dprintf
stats.c/.h	75	проверка инвариантов и итоговая статистика

Правила отправления. Диспетчер возвращает одно из трёх действий (ждать, грузить, отправлять) и причину отправления, которая попадает в журнал: паром заполнен (по числу машин, местам или массе), вышло время ожидания `max_wait`, следующая машина не влезает, очередь пуста и новых машин нет, пустой рейс за машинами на другой берег.

Стратегии формирования рейса выбираются параметром `strategy`:

- `fifo` — строго по очереди. Если первая машина не влезла, погрузка прекращается и паром уходит;
- `lookahead` — если первая машина не влезла, диспетчер просматривает следующие `window` машин и берёт первую подходящую. Одну и ту же головную машину можно пропустить не более `skip_limit` раз, после чего она должна быть погружена первой.

Приоритет экстренных служб включается параметром `priority`: экстренная машина встаёт перед обычными, но не обгоняет обычную, которая ждёт дольше `aging_limit` тактов.

Защита от бесконечного ожидания обычных машин складывается из четырёх механизмов: порядок внутри очереди FIFO; паром всегда плывёт на другой берег, поэтому берега обслуживаются по очереди; `max_wait` не даёт парому бесконечно ждать заполнения; `skip_limit` и `aging_limit` ограничивают обгон головной и давно ждущих машин.

Пользовательский интерфейс

Интерфейс консольный и построен как журнал событий: каждое значимое событие печатается отдельной строкой с отметкой времени `[t=0016]`, а в конце выводится таблица итогов. Весь вывод идёт через `dprintf` в дескрипторы `STDOUT_FILENO` и в файл журнала (`log_path`), поэтому экран и файл всегда совпадают. Пауза `delay_ms` (по умолчанию 80 мс) делается только в тактах, где что-то произошло, так что простои парома не растягивают просмотр.

Требуемое событие	Строка журнала
Прибытие автомобиля и формирование очереди	Приехала машина #7 (легковая, 1563 кг, 1 мест) на левый берег (в очереди: 2)

Требуемое событие	Строка журнала
Начало погрузки у берега	Погрузка, берег: правый (в очереди: 4)
Погрузка автомобиля	Грузим #3 (большая, 4473 кг, 3 мест) (ждала 7 мин), затем Погрузили #3 ... Паром [3/6 авто, 5/8 мест, 6825/12000 кг] #1 #2 #3
Отказ в погрузке	Отказ в погрузке: #11 (легковая, 1874 кг, 1 мест), причина: перегруз по массе
Решение диспетчера	Диспетчер пропустил #29 и грузит #18 (...) (стратегия lookahead)
Окончание погрузки	Конец погрузки (паром заполнен). Паром [6/6 авто, 8/8 мест, 10011/12000 кг] #1 #2 ...
Отправление	Отправление, рейс 1, берег отправления: левый
Прибытие парома	Паром приплыл, берег: правый, на борту 6 авто
Выгрузка	Выгружаем #1 (...), затем Выгрузили #1 (...). Паром [5/6 авто, ...]
Нарушение инварианта	НАРУШЕНИЕ ИНВАРИАНТА: ...

Текущее заполнение парома (число машин, места, масса и номера машин на борту) выводится после каждой погрузки и выгрузки, так что наблюдатель видит состояние парома в любой момент.

Параметры запуска

Все параметры задаются парами имя=значение в командной строке или в файле конфигурации (-с файл.cfg, строки вида имя = значение, # — комментарий). Аргументы командной строки обрабатываются слева направо, поэтому они переопределяют файл, если стоят после -с.

Параметр	По умолчанию	Смысл
capacity_kg	12000	грузоподъёмность парома, кг
slots	8	вместимость парома в местах
max_cars	6	максимальное число автомобилей
cars_left, cars_right	12	число автомобилей на берегу (-1 — без ограничения)
initial_left, initial_right	3	сколько машин стоит в очереди с самого начала

Если параметры неверны (не число, неизвестное имя, самая тяжёлая машина не помещается на паром, нелепые интервалы), программа печатает причину в `stderr` и завершается с кодом 2, не начиная моделирования.

Завершение зависит от исходных данных. Условия «все перевезены» и «число рейсов» проверяются, когда паром открывается для погрузки, то есть на борту в этот момент нет машин; лимит времени и прерывание проверяются в конце каждого такта.

Условие	Когда наступает	Причина в итогах	Код
Все автомобили перевезены	заданные <code>cars_left+cars_right</code> конечны и все выгружены	все автомобили перевезены	0
Лимит рейсов	задан <code>max_trips > 0</code> и выполнен	выполнено заданное число рейсов	0
Лимит времени	задан <code>max_time > 0</code> и достигнут	вышло время (<code>max_time</code>)	0
Прерывание пользователем	Ctrl+C или SIGTERM; единственный выход при <code>cars_*=-1</code> без лимитов	прервано пользователем (Ctrl+C)	0
Нет автомобилей	обе очереди изначально нулевые	машин нет	0
Нарушен инвариант	<code>check_invariants</code> нашёл ошибку	нарушен инвариант	3
Неверные параметры	до начала моделирования	сообщение в <code>stderr</code>	2

Обработчик сигнала только ставит флаг, а цикл сам выходит на границе такта, поэтому при любом способе остановки программа выводит итоги, закрывает журнал и освобождает память. Машины, которые не успели переехать, попадают в строку «Не перевезено» с разбивкой: очереди слева и справа, на пароме, ещё не приехавшие.

Инварианты

Функция `check_invariants` вызывается после каждого такта и сверяет состояние модели с пятью инвариантами из условия. При нарушении в журнал пишется НАРУШЕНИЕ ИНВАРИАНТА, моделирование останавливается, код возврата — 3.

Инвариант из условия	Как проверяется
Грузоподъёмность и вместимость не	перед посадкой <code>ferry_fits</code> сравнивает массу, места и число машин с лимитами; после такта

Инвариант из условия	Как проверяется
превышены	масса и места на борту пересчитываются заново и сверяются с накопленными значениями
Машина — либо на берегу, либо на пароме	у каждой машины поле положения (LOC_QUEUE, LOC_FERRY, LOC_DONE); при обходе борта и обеих очередей каждая машина помечается номером такта, и повторная пометка означает дубль
Движение — только после погрузки и закрытия аппарели	в состоянии «рейс» аппарель должна быть закрыта и ни одна машина не должна грузиться
Погрузка и выгрузка — только у своего берега	берег появления машины сравнивается с берегом парома (при выгрузке — с противоположным)
Машина перевозится не более одного раза	счётчик rides у машины на борту равен 1, у машины в очереди равен 0

В ходе всех запусков из раздела «Тесты» нарушений инвариантов не зафиксировано.

Тесты и результаты работы

Наборы данных лежат в tests/*.cfg, запуск — make test (скрипт run_tests.sh, результат [OK]/[FAIL] по каждому набору, логи в logs/). Скрипт автоматически проверяет все семь наборов (basic, lookahead, emergency, one_side, trips_limit, invalid, unlimited), сейчас для них всегда [OK]. Во всех прогонах нарушений инвариантов нет.

Набор	Что проверяет	Результат
basic	10+10 машин, параметры по умолчанию, seed=11	все 20 перевезены за 160 мин, 5 рейсов, 1 отказ; ожидание обычных 47,9 мин
lookahead	грузоподъёмность 9000 кг, тяжёлые машины, 15+15, стратегия lookahead	все 30 перевезены за 270 мин, 10 рейсов (1 пустой), 10 отказов; диспетчер пропускает неподходящую машину и берёт следующую
emergency	20+20 машин, 40% экстренных	все 40 перевезены за 324 мин, 10 рейсов; ожидание экстренных 67,7 мин против 125,8 мин у обычных

Набор	Что проверяет	Результат
one_side	машины только слева (9 штук)	все 9 перевезены за 81 мин, 3 рейса, из них 1 пустой — паром возвращается порожним
trips_limit	40+40 машин, max_trips=4	остановка после 4 рейсов за 128 мин: перевезено 16, не перевезено 64 (в очередях 18+19, ещё не приехали 27)
invalid	capacity_kg=3000 — тяжёлая машина не помещается	сообщение об ошибке, код возврата 2, моделирование не начинается
unlimited	cars_*=-1, остановка по SIGINT через 2 с	корректное завершение с итогами; за 2449 мин проехало 305 машин, в очередях осталось 354+336

Фрагмент журнала (basic, начало первого рейса)

```
[t=0000] Паром у берега: левый, начинаем погрузку
[t=0000] Приехала машина #1 (легковая, 1512 кг, 1 мест) на левый берег (в очереди: 1)
[t=0001] Грузим #1 (легковая, 1512 кг, 1 мест) (ждала 1 мин)
[t=0003] Погрузили #1 (легковая, 1512 кг, 1 мест). Паром [1/6 авто, 1/8 мест, 1512/12000 кг] #1
[t=0018] Погрузили #11 (легковая, 884 кг, 1 мест). Паром [6/6 авто, 8/8 мест, 10011/12000 кг] #1 #2 #3 #7 #8 #11
[t=0019] Конец погрузки (паром заполнен). Паром [6/6 авто, 8/8 мест, 10011/12000 кг] #1 #2 #3 #7 #8 #11
[t=0020] Отправление, рейс 1, берег отправления: левый
[t=0030] Паром приплыл, берег: правый, на борту 6 авто
[t=0031] Выгружаем #1 (легковая, 1512 кг, 1 мест)
[t=0032] Выгрузили #1 (легковая, 1512 кг, 1 мест). Паром [5/6 авто, 7/8 мест, 8499/12000 кг] #2 #3 #7 #8 #11
```

Фрагмент журнала (lookahead, отказ и решение диспетчера)

```
[t=0072] Отказ в погрузке: #29 (экстренная, 3924 кг, 2 мест), причина: перегруз по массе
[t=0072] Диспетчер пропустил #29 и грузит #18 (легковая, 1981 кг, 1 мест)
```

Итоговая статистика (emergency)

```
Итоги:
Причина завершения: все автомобили перевезены
Время: 324 мин
Рейсов: 10 (пустых: 0)
Отказов в погрузке: 4
Перевезено машин: 40 (слева 20, справа 20)
```

Не перевезено: 0 (в очередях 0+0, на пароме 0, не приехали 0)
Среднее ожидание: обычные 125.8 мин, экстренные 67.7 мин

Промпты к ИИ

Каркас программы (структуры, очереди, паром и главный цикл) я написал сам; ИИ-ассистент Claude использовался для доработки: статистика, тесты, сокращение кода и комментариев. Я ставил задачи, решал, что оставить, а что сократить, и проверял результат запуском. Промпты приведены в хронологическом порядке; формулировки слегка сокращены, смысл сохранён.

№	Промпт	Что получилось
1	привет, вот условие лабы (вариант 28, паромная переправа) и методичка. Помоги мне написать вывод статистики	Функция <code>stats_print</code> , запись в лог-файл через <code>dprintf</code>
2	сделай какие-нибудь тесты и скрипт для запуска по требованиям	Семь файлов <code>tests/*.cfg</code> и <code>run_tests.sh</code>
3	тесты падают	Проверки искали не те фразы; приведены к фактическим сообщениям журнала
4	добавь мне комментарии в код	Добавление комментариев
5	комменты в коде слишком подробные, сделай проще, человечнее и короче	Комментарии сокращены
6	сообщения, которые программа выводит слишком длинные, сделай базовый минимум	Сообщения журнала стали короче
7	упрости тесты	Тестовые конфиги сведены к 2–7 строкам
8	давай теперь отчёт по методичке: ФИО, группа, вариант, условие, сценарий, интерфейс, запуск и завершение, результаты. и промпты с критической оценкой, цифры бери из	Этот отчёт

№	Промпт	Что получилось
	реальных запусков	

Критическая оценка и правки

ИИ помог с выводом статистики, тестами и комментариями, но результат пришлось проверять и править: часть проверок в тестах была неверной, а комментарии и сообщения — слишком подробными. Ниже — что удалось, что пришлось исправлять и что осталось недоработанным.

Что удалось

- Паром, диспетчер и автомобили выделены в отдельные модули со своим состоянием, как требует условие.
- После каждого такта проверяются все пять инвариантов, а не только один раз в конце.
- Ожидание обычных машин ограничено четырьмя механизмами (FIFO, чередование берегов, `skip_limit` и `aging_limit`); на тесте `emergency` приоритет реально работает: 67,7 мин ожидания у экстренных против 125,8 мин у обычных, при этом обычные всё равно уезжают.
- Программа завершается корректно во всех трёх случаях (по данным, по лимитам, по Ctrl+C) и всегда печатает статистику с неперебазированными машинами.

Что пришлось исправлять после генерации

1. В первой версии `stats_print` считал неперебазированных машин только по очередям; добавили машины на пароме и ещё не приехавшие, иначе строка «Не перевезено» не соответствовала условию.
2. В `run_tests.sh` часть проверок искала не те фразы и падала, хотя программа работала правильно; поправили под реальный вывод.
3. Комментарии в коде сначала получились слишком подробными и «книжными»; переписали проще и короче, чтобы они читались как мои собственные.
4. Сообщения, которые выводит программа, были длинными — оставили базовый минимум, необходимый по методичке.
5. Тестовые конфиги были громоздкими — свели до 2–7 строк на набор.

Вручную после проверки проекта я добавил в `run_tests.sh` наборы `one_side` и `trips_limit` (теперь автоматически проверяются все семь) и удалил неиспользуемый файл `src/ferry_sim.h`, оставшийся от черновой версии, вместе с его зависимостью в `Makefile`.

Что осталось недоработанным

Проблема	Почему это проблема
При параметрах по умолчанию режим «без ограничения» перегружен	Паром везёт около 6 машин за рейс длительностью около 30 мин, а с двух берегов в сумме приезжает в среднем одна машина в 2,5 минуты — поток вдвое больше пропускной

Проблема	Почему это проблема
	способности. В тесте <code>unlimited</code> за 2449 мин очереди выросли до 354 и 336 машин, среднее ожидание — около 840 мин. Формально каждая машина рано или поздно поедет (очередь FIFO), но ожидание растёт без предела. Для устойчивого режима нужны более редкие прибытия (<code>arr_min</code> , <code>arr_max</code>).
Приоритет экстренных исчезает при длинных очередях	Когда все ждут дольше <code>aging_limit</code> (30 мин), экстренная машина никого не обгоняет. В тесте <code>unlimited</code> среднее ожидание экстренных (857 мин) не лучше, чем обычных (841 мин). Это цена защиты от голодания, но правило приоритета так становится формальным.
Распределение размеров машин зашито в код	Доли 60/20/10/10 % заданы в <code>car_spawn</code> ; в условии размер — входной параметр, а пользователь задаёт только верхнюю границу <code>size_max</code> .
Проверка параметров неполная	При упрощении кода проверки оставлены на минимуме: не проверяются <code>strategy</code> (любое число, кроме 1, трактуется как <code>fifo</code>), <code>emergency_pct</code> вне 0–100 и отрицательные <code>delay_ms</code> и <code>max_trips</code> ; отрицательная пауза может дать очень долгую задержку. Файл настроек читается одним <code>read</code> не более 8 КБ.
Тесты проверяют мало	<code>run_tests.sh</code> ищет только фразу «все автомобили перевезены» (или причину остановки) и отсутствие «НАРУШЕНИЕ»; конкретные числа в статистике не сверяются.
Пустой паром может ждать без нужды	Если на текущем берегу очередь пуста, но машины ещё будут приезжать, паром до <code>max_wait</code> (20 мин) стоит пустым, даже если на другом берегу уже есть очередь.
«Самостоятельные участники» — условно	Всё выполняется в одном потоке; паром — конечный автомат, а диспетчер — функция без собственного состояния. Для первого задания (последовательная программа) это допустимо, но настоящей конкурентности в ней нет.

Проблема	Почему это проблема
Мелкие недочёты вывода	«1 мест», «ждала 0 мин» — грамматика сообщений не учитывает числители; время измеряется тактами, а не реальными секундами.

Выводы

Программа моделирует паромную переправу на языке С: паром, диспетчер и автомобили выделены в отдельные модули, журнал событий показывает всё, что требует условие, а инварианты проверяются после каждого такта. Все семь наборов данных завершаются штатно, неперебазённые машины попадают в итоговую статистику. Главные слабые места — перегрузка в режиме без ограничения при параметрах по умолчанию, формальный приоритет экстренных при длинных очередях и скромные автоматические тесты; они перечислены в разделе критической оценки.